

Pilote Automatique F1 Virtuel

Clemente Iwan — Picard Marius — Moulin Yann

Décembre 2025

- 1 Introduction — Contexte et objectifs du projet
- 2 Architecture & Algorithme — Structure modulaire, A* 4D
- 3 Points clés — Heuristique, carburant, optimisations
- 4 Bilan — Résultats, difficultés, retour d'expérience
- 5 Conclusion — Synthèse et perspectives

1. Introduction — Contexte du projet

Présentation du projet

- Concevoir un pilote autonome en C
- Interaction temps réel avec le GDC (stdin/stdout)
- Réponse obligatoire en moins de 1 seconde par tour
- Trois voitures en compétition simultanée

Objectifs & contraintes

- Atteindre l'arrivée le plus vite possible
- Ne pas sortir de piste, ni tomber en panne sèche
- Vitesse limitée sur sable : $|v| \leq 1$
- C99 strict : -ansi -pedantic -Wall -Wextra
- Vitesse max : norme ≤ 5

Terrains — ".": route "~": herbe "=": sable (lent) "#": mur (crash)

2. Architecture — Structure modulaire

8 modules indépendants en C99

main.c

Point d'entrée, boucle de jeu

map.c

Parsing et stockage de la carte

path_search.c

A* 4D — recherche de trajectoire

heap.c

Tas binaire pour la file de priorité

communication.c

Lecture/écriture stdin/stdout (GDC)

dijkstra.c

Pré-calcul de l'heuristique

physic.c

Modèle physique (vitesse, carburant)

prio_queue.c

File de priorité à nœuds pré-alloués

2. Algorithme — A* dans l'espace 4D

Pourquoi un espace d'état 4D ?

- La voiture possède de l'inertie
- Même case (x, y) : comportement différent selon (vx, vy)
- État = (x, y, vx, vy) indispensable
- Heuristique : Dijkstra inversé pré-calculé depuis l'arrivée
- Sable pénalisé (+7) → évitement naturel
- Coût $g \times 7$ pour homogénéité avec h

Cycle de décision (chaque tour)

- ① Lecture de la position et vitesse (GDC)
- ② A* 4D — exploration jusqu'à 0,9 s CPU
- ③ Sélection du meilleur mouvement
- ④ Validation physique (follow_line)
- ⑤ Envoi de dvx, dvy au GDC

 Timeout interne 0,9 s : le meilleur chemin partiel est toujours renvoyé, jamais de dépassement.

3. Points clés — Optimisations techniques

› Pré-allocation des nœuds

Aucun malloc() pendant la course. Le pool de nœuds A* est alloué une seule fois au démarrage. Élimine la fragmentation mémoire et les appels système coûteux.

› Marquage 4D par identifiant de recherche

Au lieu de réinitialiser un tableau à chaque tour, un ID de recherche change à chaque itération. Réinitialisation en $O(1)$ plutôt qu' $O(n^4)$.

› Meilleur carburant par état

Un état est écarté si on l'a déjà atteint avec plus de carburant. Intégré dans le marquage 4D, sans coût supplémentaire.

› Validation physique follow_line()

Vérifie toutes les cases intermédiaires d'un mouvement. Garantit qu'aucune case hors-piste n'est "sautée" à grande vitesse.

Marquage 4D par identifiant de recherche

Le problème

L'espace d'état 4D contient jusqu'à ~2,4 M de cases (x, y, vx, vy). Réinitialiser un tableau de cette taille à chaque tour coûte trop cher en temps.

La solution

Chaque case stocke un entier : **search_id**.

Au lieu de tout remettre à zéro, on incrémente juste **search_id** d'une unité.

Une case est considérée
« **non visitée** » si sa valeur $\neq$ search_id courant.

Complexité

Réinit. classique : $O(n^4) \rightarrow \sim 2,4 \text{ M}$ opérations

Avec search_id : $O(1) \rightarrow 1$ seule opération

Illustration Tour N vs Tour N+1

Tour N (search_id = 5) de l'état (x, y, vx, vy)

(2, 3, 1, 0) id=5	(2, 3, 0, 1) id=5	(2, 3, 1, 1) id=3
(3, 3, 1, 0) id=5	(3, 3, 2, 0) id=5	(3, 3, 0, 2) id=2
(4, 3, 2, 0) id=5	(4, 3, 1, 1) id=1	(4, 3, 2, 1) id=5

 id=5 $\rightarrow$ visité ce tour

 id $\neq$ 5 $\rightarrow$ non visité

$\rightarrow$ **search_id++** (on passe à 6)

Tour N+1 (search_id = 6) — toutes les cases sont instantanément « fraîches »

Les cases contiennent encore 5, 3, 2, 1... mais search_id = 6 $\rightarrow$ aucune n'est considérée visitée.

Aucune boucle de remise à zéro. Coût : $O(1)$.

4. Bilan — Résultats obtenus

Points forts

- Finit toutes les courses testées
- Ne sort jamais de la piste
- Gestion correcte des collisions
- Respect des limites de vitesse (sable)
- Temps de réponse $< 0,9$ s, toutes cartes
- Trajectoires propres (virages, lignes droites)

Limites identifiées

- Boosts non utilisés (stratégie abandonnée)
- Adversaires ignorés dans la planification
- Comportement conservateur si carburant bas
- Sous-optimal sur longues lignes droites

4. Bilan — Difficultés rencontrées

Gestion du carburant

Problème : Le pilote terminait en panne sèche sur les grandes cartes.

Solution : Stockage du meilleur niveau de carburant par état 4D. Un état plus coûteux en carburant est systématiquement écarté.

Coût temporel du A* 4D

Problème : L'allocation de nœuds à chaque tour dépassait la limite d'1 seconde.

Solution : Pool pré-alloué + marquage par ID de recherche + timeout interne à 0,9 s.

Intégration des boosts

Problème : Tentative : $dv_x/dv_y \in [-2, 2]$. Le pilote consommait trop tôt → moins performant.

Solution : Décision prise de ne pas utiliser les boosts pour garantir la stabilité.

4. Bilan — Organisation et retour d'expérience

Répartition du travail

- Iwan Clemente : implémentation de l'algorithme A^*
- Marius Picard : Dijkstra inversé (calcul de l'heuristique)
- Yann Moulin : communication GDC, file de priorité

Ce que nous avons appris

- La physique doit être dans l'algorithme dès le départ — pas en post-traitement
- L'approche incrémentale (tester souvent contre le GDC) accélère la stabilisation
- L'IA générative est utile sur les parties techniques, mais la relecture du code reste indispensable
- Un compromis explicite (ex : abandon des boosts) vaut mieux qu'une solution fragile

5. Conclusion

Synthèse

Notre pilote repose sur une recherche A^* dans un espace d'état 4D (x, y, vx, vy), guidée par une heuristique Dijkstra inversée pré-calculée. Il finit toutes ses courses, respecte les contraintes physiques et temporelles, et reste modulaire et évolutif.

Perspectives d'amélioration

- Intégration des boosts avec déclenchement conditionnel
- Heuristique 4D intégrant la vitesse → moins de nœuds à explorer
- Prise en compte des mouvements adversaires sur N tours